

What Changed (Quick Summary)

GET /api/v1/booking/my	Returned booking list — no Agora details	Each Confirmed / Active booking now includes agoraInfo (channelName, appId, agoraUid)
GET /api/v1/booking/healer/ongoing	Returned only Active sessions — empty when nothing in progress	If no ongoing session, returns the next upcoming Confirmed session so healer can prepare to join

Change 1 — GET /api/v1/booking/my GET

Used for the My Bookings screen (Patient or Healer). Every Confirmed or Active booking now includes an agoraInfo block with everything Flutter needs to join the Agora channel.

Request

```
// Patient view (default)
GET /api/v1/booking/my?isHealer=false
Authorization: Bearer <token>
```

```
// Healer view
GET /api/v1/booking/my?isHealer=true
Authorization: Bearer <token>
```

Response — Each Booking Item

```
{
  "id": "booking-uuid",
  "bookingReference": "HU-20250618-ABC123",
  "status": "Confirmed",
  "serviceType": "Personal",
  "bookingDate": "2025-06-18",
  "scheduledStartTime": "14:00:00",
  "scheduledEndTime": "15:00:00",
```

```
// — NEW FIELD
```

```
"agoraInfo": {
  "channelName": "humora-hu-20250618-abc123", // pass to Agora SDK
  "appId":     "your_agora_app_id",           // pass to Agora SDK init
  "agoraUid":  "1234567890"                   // pass to joinChannel uid
}
```

```
// agoraInfo is NULL for Completed / Cancelled / Expired bookings
}
```

Important: agoraInfo is null for Completed, Cancelled, and Expired bookings. Always null-check before using it.

How Flutter Uses This

Load the booking list by calling GET /booking/my.

Show a Join button only when agoraInfo != null.

On tap, call POST /agora/token with bookingId to get the RTC token.

Initialize Agora SDK with appId, join channel with channelName, agoraUid, and the RTC token.

Flutter Code

```
class AgoraInfo {
  final String channelName, appId, agoraUid;
  AgoraInfo.fromJson(Map<String, dynamic> j)
    : channelName = j['channelName'],
      appId      = j['appId'],
      agoraUid   = j['agoraUid'];
}

// On "Join" button tap
Future<void> onJoinTapped(BookingItem booking) async {
  final info = booking.agoraInfo;
  if (info == null) return; // not joinable

  // 1. Get RTC token from server
  final tokenResp = await api.post('/agora/token', body: {
    'bookingId': booking.id,
    'role': 'Publisher',
  });

  // 2. Init SDK and join
  await RtcEngine.createWithContext(RtcEngineContext(info.appId));
  await engine.joinChannel(
    token: tokenResp['token'],
    channelId: info.channelName,
    uid: int.parse(info.agoraUid),
    options: const ChannelMediaOptions(),
  );
}
```

Change 2 — GET /api/v1/booking/healer/ongoing GET

Only for healer and show this in dashboard proper create ui like my app theme

Used on the Healer Dashboard / Home screen. Now handles 3 scenarios instead of 2.

Request

GET /api/v1/booking/healer/ongoing
Authorization: Bearer <token>

Scenario A — Healer Has Active Sessions

Same as before. Each session item now includes agoraInfo. isUpcomingFallback is false.

```
{
  "totalOngoing": 1,
  "isUpcomingFallback": false,
  "upcomingSession": null,
  "personalBookings": [
    {
      "bookingId": "uuid",
      "serviceType": "Personal",
      "actualStartAt": "2025-06-18T14:02:00Z",
      "elapsedSeconds": 480,
      "patient": { "fullName": "Raj Kumar" },
      "agoraInfo": { // ← NEW
        "channelName": "humora-hu-20250618-abc123",
        "appId": "your_agora_app_id",
        "agoraUid": "9876543210"
      }
    }
  ],
  "liveConsultations": [],
  "groupBookings": []
}
```

Scenario B — No Active Sessions, Next Upcoming Found NEW

When no session is in progress, isUpcomingFallback: true and upcomingSession is the next scheduled booking. Show a countdown to scheduledStartAt.

```
{
  "totalOngoing": 0,
  "isUpcomingFallback": true, // ← check this flag
  "upcomingSession": {
    "bookingId": "uuid",
    "serviceType": "Personal",
    "scheduledStartAt": "2025-06-18T16:00:00Z", // show countdown to this
    "actualStartAt": null,
    "patient": { "fullName": "Priya Shah" },
    "agoraInfo": {
      "channelName": "humora-hu-20250618-xyz789",
      "appId": "your_agora_app_id",
      "agoraUid": "9876543210"
    }
  }
}
```

```

    },
    "liveConsultations": [],
    "personalBookings": [],
    "groupBookings": []
  }
}

```

Scenario C — Nothing At All

```

{
  "totalOngoing": 0,
  "isUpcomingFallback": false,
  "upcomingSession": null,    // healer has nothing scheduled
  "liveConsultations": [],
  "personalBookings": [],
  "groupBookings": []
}

```

Flutter Dashboard Logic

```

void renderHealerDashboard(HealerOngoingResponse resp) {
  if (resp.totalOngoing > 0) {
    // Scenario A — show active session cards with elapsed timer
    final all = [
      ...resp.liveConsultations,
      ...resp.personalBookings,
      ...resp.groupBookings,
    ];
    showActiveSessions(all);

  } else if (resp.isUpcomingFallback && resp.upcomingSession != null) {
    // Scenario B — show countdown card to next session
    showUpcomingCard(resp.upcomingSession!);

  } else {
    // Scenario C — nothing scheduled
    showEmptyState("No sessions today");
  }
}

// When healer taps "Start" on the upcoming card
Future<void> onStartSession(OngoingSession session) async {
  final tokenResp = await api.post('/agora/token', body: {
    'bookingId': session.bookingId,
    'role': 'Publisher',
  });
  await engine.joinChannel(
    token: tokenResp['token'],
    channelId: session.agoraInfo!.channelName,
    uid: int.parse(session.agoraInfo!.agoraUid),
  );
}

```

```

    options: const ChannelMediaOptions(),
  );
}

```

Tip: When `isUpcomingFallback` is true, show a countdown timer to `upcomingSession.scheduledStartAt`. The `agoraInfo` is already included — the healer can tap Start the moment time arrives with no extra API calls.

Getting the RTC Token (Required Before Joining)

`agoraInfo` does not include the RTC token — tokens expire in 1 hour and must be generated on demand. Use the info to call the token endpoint first.

POST /api/v1/agora/token

// Request body

```

{
  "bookingId": "booking-uuid",
  "role": "Publisher" // Healer and Patient both use "Publisher"
}

```

// Response

```

{
  "token": "006yourAgoraRTCtoken...",
  "channelName": "humora-hu-20250618-abc123",
  "appId": "your_agora_app_id",
  "agoraUid": "1234567890",
  "expiresInSeconds": 3600,
  "expiresAt": "2025-06-18T15:00:00Z"
}

```

Token Refresh: Refresh the token 5 minutes before `expiresAt` by calling the token endpoint again with the same `bookingId`. Use `engine.renewToken(newToken)` — no need to rejoin the channel.

agoraInfo Field Reference

channelName	string	Pass as <code>channelId</code> to <code>engine.joinChannel()</code>
appId	string	Pass to <code>RtcEngine.createWithContext(RtcEngineContext(appId))</code>
agoraUid	string (numeric)	Pass as <code>uid</code> : <code>int.parse(agoraUid)</code> to <code>engine.joinChannel()</code> . Same UID for this user across all sessions.

Flutter Developer Checklist

--	--

Add AgoraInfo model	Parse channelName, appId, agoraUid from JSON. Make it nullable in booking model.
Show Join button conditionally	Only show Join button when agoraInfo != null (status is Confirmed or Active)
On Join/Start tap	Call POST /agora/token → get token → init SDK with appId → join with channelName, agoraUid, token
Parse isUpcomingFallback	Add boolean field to HealerOngoingResponse model. Default false.
Parse upcomingSession	Add nullable OngoingSession? upcomingSession to response model.
Healer dashboard 3-state render	Active sessions → show cards. Upcoming fallback → show countdown card. Neither → show empty state.
Token refresh	Refresh 5 min before expiresAt. Call POST / agora/token again, then engine.renewToken().